Completion check week 7: part 1

a.

```
public void printSomePoints(List<Point> points) {

    int printedPoints = 0;
    for (Point point : points) {
        if (someCondition()) {
            System.out.println(String.format("Point %d (%f,%f)",
                    printedPoints, point.getX(), point.getY()));
        }
    }
}

class Point {
    private double x;
    private double y;

    //getters and setters
}
```

This does not violate LoD because the method is using methods invoked by its parameters.

b.

```
public void printDistanceBetweenRandomPointsOfTwoSquares(Square
s1, Square s2) {
    Point randomPoint1 = s1.getRandomPoint();
    Point randomPoint2 = s2.getRandomPoint();

    System.out.println(randomPoint1.distanceTo(randomPoint2));
}

@AllArgsConstructor
class Point {
    @Getter private double x;
    @Getter private double y;

    public double distanceTo(Point p){
        return Math.hypot(x-p.x, y-p.y);
    }
}
```

[redacted]

```
@Data //getters and setters
@AllArgsConstructor
class Square {
    private Point[] points = new Point[4];

    public Point getRandomPoint(){
        int index = (int) (Math.random()*4);
        return points[index];
    }
}
```

This does not violate LoD neither as it only interacts with the Square class, and since the method as to Square objects as parameters, it is okay.

c.

```
public void someMethod(Student student) {
   Faculty faculty = student.getFaculty();

   System.out.println(faculty.getName());
   System.out.println(faculty.getStudentNo());
}

@Data
@AllArgsConstructor
class Student {
   private String name;
   private Faculty faculty;

}

@Data
@AllArgsConstructor
class Faculty {
   private List<Student> students;

   public int getStudentNo() { return students.size(); }
   public void removeStudent(Student student) {
students.remove(student); }
   public void addStudent(Student student) {
students.add(student); }
}
```

This is also okay, since in the method block there are invoked methods over the parameters and over the objects created in the method.

d.

```
public void someMethod() {
    //...some code
    Student student = Student.builder()
            .lastName("LN")
            .firstName("FN")
            .age(20)
            .bestFriend(null)


            .build();
      System.out.println(student);
      //...some other code
   }

   @AllArgsConstructor
   class Student {
      private String firstName;
      private String lastName;
      private Integer age;
      private Student bestFriend;

      public static StudentBuilder builder() {
          return new StudentBuilder();
      }
   }

   class StudentBuilder {
      private String firstName;
      private String lastName;
      private Integer age;
      private Student bestFriend;

      StudentBuilder() { }

      public StudentBuilder firstName(String firstName) {
   this.firstName = firstName; return this; }

      public StudentBuilder lastName(String lastName) {
   this.lastName = lastName; return this; }

      public StudentBuilder age(Integer age) { this.age = age;
   return this; }

      public StudentBuilder bestFriend(Student bestFriend) {
   this.bestFriend = bestFriend; return this; }

      public Student build() { return new Student(firstName,
   lastName, age, bestFriend); }
   }
```

This method calls are all on directly created local object because Student.builder() returns a StudentBuilder, so it's legal.

2. A data structure is a container for storing data while a class encapsulates both data and behavior. LoD applies to classes because its main goal is to limit how objects communicate and preserve encapsulation.

Completion check week 7: part 2

1. What is the contract of the following method? public static Collection<String> readLinesFromFile(String absoluteFilePath)

   Preconditions: absoluteFilePath is non-null and refers to a readable file

   Postconditions: the method returns a non-null collection containing all the data from the file passed as paramter

   Invariants: the content of the file stays the same. The method will not change any global or static variables.

   Side effects: the method may throw some runtime exceptions (IOException, FIleNotFoundException). The method will also allocate memory for the data from the file.

2. What principle is violated in the following example? Explain your answer. How would you refactor the code to not violate the principle?

```
interface IMembership {
   void login(UserCredentials uc);
   boolean logout();
   UserCredentials register(String userName, String password);
   UserCredentials changePassword(String userName);
   void addPaymentMethod(UserCredentials uc, PaymentMethod pm);
}
```

This violates the Interface segregation principle. Example why this happens: a class will only want to work with the user credentials, but it will be forced to also implement the login logout methods and deal with the payment method. A fix for this code will be dividing the interface in more, smaller interfaces:

```
interface IAuthentication {
   void login(UserCredentials uc);
   boolean logout();
}

interface IAccountManagement {
   UserCredentials register(String userName, String password);
   UserCredentials changePassword(String userName);
}

interface IPaymentManagement {
   void addPaymentMethod(UserCredentials uc, PaymentMethod pm);
}
```

3. Does the following method respect design by contract? Why? How can you ensure that the base method's contract will never be breached by a method that is overriding it?

```
String Base::aMethod(int[] args)
// Preconditions: args contains only negative numbers
// Postconditions: returns a String with only lowercase characters or digits

String Derived::aMethod(int[] args)
// Preconditions: args contains any negative or even numbers
// Postconditions: returns a String composed only of digits or null
```

The derived method violates the base contract. While its preconditions are weaker (it accepts more inputs — negative *or* even numbers), its postconditions are weaker as well — it may return `null` and no longer guarantees lowercase letters.
This breaks the principle that overriding methods must *not* weaken postconditions. In order to ensure the base contract is preserved, In overriding methods, keep preconditions the same or weaker, and postconditions the same or stronger.

4.Imagine you have a class ProductsOffer that stores a collection of Products. Each product has name, manufacturer name, color, price, weight and release date. In the first version ProductsOffer provides the next services for filtering the products:

```
class ProductsOffer {

   List<Product> products;


   List<Product> getByName(String name) { ... }

   List<Product> getByManufacturerName(String manufacturer) { ... }

   List<Product> getByColor(String color) { ... }

   List<Product> getByPrice(double price) { ... }

   List<Product> getByWeight(double weight) { ... }

   List<Product> getByReleaseDate(LocalDate date) { ... }


   // later additions:

   List<Product> getByNameAndColor(String name, String color) { ... }

   List<Product> getByWeightAndReleaseDate(double weight, LocalDate date) { ... }

}
```

The design is poor as for every new filter requirement there should be a modifications in the class, leading to duplicate code and violating the Open/Close principle.

Solution: there should be introduced a filter method which takes filtering criteria and performs the operation of filtration based on the parameters.

```
import java.util.function.Predicate;
import java.util.stream.Collectors;

class Product {  7 usages
    private final String name;  1 usage
    private final String manufacturer;  1 usage
    private final String color;  1 usage
    private final double price;  1 usage
    private final double weight;  1 usage
    private final LocalDate releaseDate;  1 usage


    Product(String name, String manufacturer, String color, double price, double weight, LocalDate releaseDate) {  no usages
        this.name = name;
        this.manufacturer = manufacturer;
        this.color = color;
        this.price = price;
        this.weight = weight;
        this.releaseDate = releaseDate;
    }

    // constructor + getters omitted for brevity
}

class ProductsOffer {  2 usages
    private final List<Product> products = new ArrayList<>();  1 usage

    public List<Product> filter(Predicate<Product> predicate) {  2 usages
        return products.stream()
                .filter(predicate)
                .collect(Collectors.toList());
    }
}

public static void main(String[] args) {
    ProductsOffer offer = new ProductsOffer();

    Predicate<Product> byName = Product p -> p.getName().equals("Phone X");
    Predicate<Product> byColor = Product p -> p.getColor().equals("Black");
    Predicate<Product> byWeight = Product p -> p.getWeight() < 0.5;
    Predicate<Product> byDate = Product p -> p.getReleaseDate().isAfter(LocalDate.of( year: 2022,  month: 1,  dayOfMonth: 1));

// Example usages:
    offer.filter(byName.and(byColor));
    offer.filter(byWeight.and(byDate));
}
```

5. What principle is violated in the following example? Explain why. How would you refactor the code to not violate the principle?

```
abstract class Vehicle {
   private int peopleCapacity;

   abstract double getSpeed();
   abstract double getGasLevel();
   abstract void pressClutch();
}

   class Scooter extends Vehicle{
         //does not have a clutch
}
```

```
class Tesla extends Vehicle{
      //Tesla is an Electric Car
      int getTrunkCapacity(){...}
      int getFrunkCapacity(){...}
      double getAutonomy(){...}
}

class Logan extends Vehicle{
      int getTrunkCapacity(){...}
}
```

This code violates the Liskov substitution principle because the class Vehicle tries to represent too many types of vehicles. In this manner, not all subclasses can implement the methods from the base class in a meaningful way. Example: scooters don't have a clutch, tesla does not have a gas level.
A solution to this would be to implement the appropriate interfaces and divide the code by taking into account all types of vehicles we want to represent, each of them having to implement a designed interface:

```
abstract class Vehicle {  3 usages
    private int peopleCapacity;  no usages
    abstract double getSpeed();  no usages
}

interface FuelPowered {  2 usages
    double getGasLevel();  no usages
}

interface ElectricPowered {  1 usage
    double getBatteryLevel();  no usages
    double getAutonomy();  no usages
}

interface ManualTransmission {  1 usage
    void pressClutch();  no usages
}
class Scooter extends Vehicle implements FuelPowered {  no usages
    @Override double getSpeed() { ... }  no usages
    @Override public double getGasLevel() { ... }  no usages
}

class Tesla extends Vehicle implements ElectricPowered {  no usages
    @Override double getSpeed() { ... }  no usages
    @Override public double getBatteryLevel() { ... }  no usages
    @Override public double getAutonomy() { ... }  no usages
}

class Logan extends Vehicle implements FuelPowered, ManualTransmission {  no usages
    @Override double getSpeed() { ... }  no usages
    @Override public double getGasLevel() { ... }  no usages
    @Override public void pressClutch() { ... }  no usages
}
```

6. What principle is violated in the following example? Explain why. How would you refactor the code? Draw a class diagram showing your solution.

```
class Student {
  …
  public void saveStudent();
  public Double calculateMeanGrade();
  public Profile getStudentProfile();
}
```

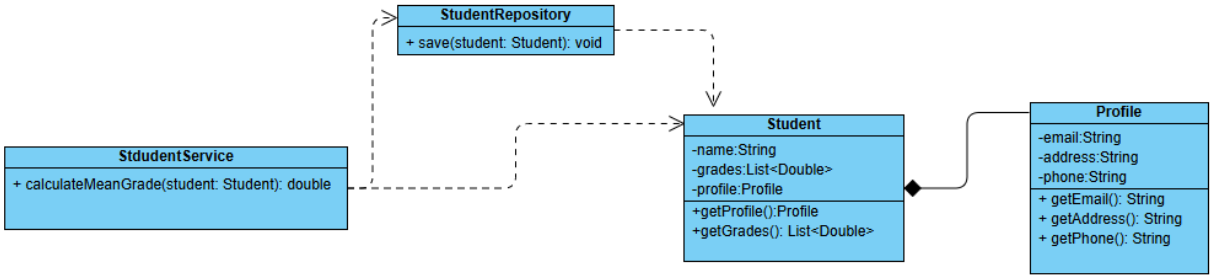
This violates the single responsibility principle. The Student class currently handles data storage, persistence, and business logic. Each of these responsibilities can change independently, giving the class multiple reasons to change.

Solution: split responsibilities:

Student holds the domain data.

StudentRepository handles saving/loading students.

StudentService handles calculations like mean grade.




```java
import java.util.List;

class Profile {  3 usages
    private String email;  2 usages
    private String address;  2 usages
    private String phone;  2 usages

    public Profile(String email, String address, String phone) {  no usages
        this.email = email;
        this.address = address;
        this.phone = phone;
    }

    public String getEmail() { return email; }  no usages  4 related problems
    public String getAddress() { return address; }  no usages
    public String getPhone() { return phone; }  no usages
}

class Student {  74 usages  7 related problems
    private String name;  2 usages
    private List<Double> grades;  2 usages
    private Profile profile;  2 usages

    public Student(String name, List<Double> grades, Profile profile) {  no usages  7 related problems
        this.name = name;
        this.grades = grades;
        this.profile = profile;
    }

    public String getName() { return name; }
    public List<Double> getGrades() { return grades; }  1 usage
    public Profile getProfile() { return profile; }  no usages
}
```

```java
class StudentRepository {  4 usages
    public void save(Student student) {  1 usage
        // persist student to database
        System.out.println("Saving student: " + student.getName());
    }

    public Student load(String name) {  no usages
        // load student from database (dummy example)
        return null;
    }
}

class StudentService {  no usages
    private StudentRepository repository;  2 usages

    public StudentService(StudentRepository repository) {  no usages
        this.repository = repository;
    }

    public double calculateMeanGrade(Student student) {  no usages
        List<Double> grades = student.getGrades();
        return grades.stream().mapToDouble(Double::doubleValue).average().orElse( other: 0.0);
    }

    public void saveStudent(Student student) {  no usages
        repository.save(student);
    }
}
```